面试笔记：WebGen 项目工程点

面试笔记：从 WebGen 长期记忆项目里能讲的工程点

> 用途：准备面试（偏后端 / Java 方向）。每个知识点都来自这个项目里真实做过的事，讲的时候按”问题 → 方案 → 取舍 → 结果”来讲。 维护方式：项目里以后遇到值得讲的工程问题，就在对应章节后面追加。最后更新：2026-10-07。

---

1. 在共享 GPU 集群上自建”长驻工作者 + 动态任务队列”

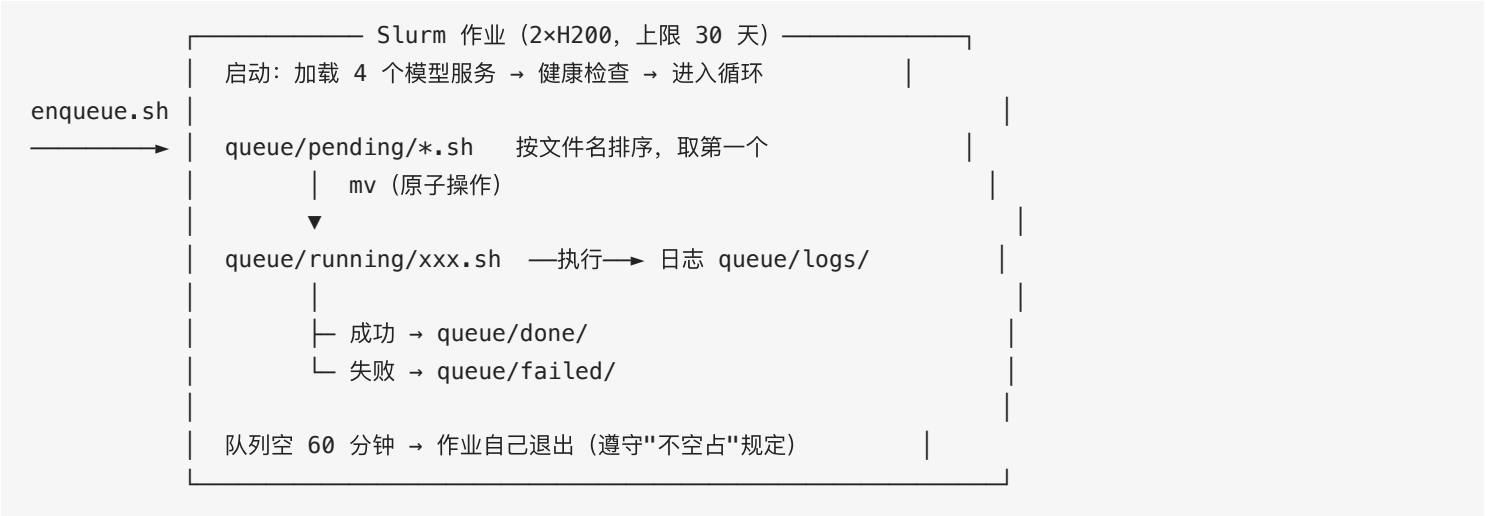
1.1 遇到的问题

学校的 GPU 集群用 Slurm 调度：一个节点 7 张 H200，十几个人共用。

- **排队很久**：每提交一个作业都要重新排队，别人的作业可能插在前面。我们曾经因为取消后重交，丢了排队位置。
- **启动很慢**：每个作业要先把 4 个模型（7B 生成、32B 评测、72B 记忆、embedding）加载到显卡上，要 3–4 分钟；如果一个实验一个作业，这些时间会反复浪费。
- **个人上限**：每人同时最多占大约 2 张卡，超了作业会被挂起（`JobHeldUser`）。
- **集群规定**：不允许为了”让模型一直开着”而空占显卡。
- **实验会变**：跑到一半常常要换方案、插一个诊断、调整顺序，不能每次都停作业重交。

1.2 方案：一个作业拿到卡，自己在里面按队列干活

这种做法在业界叫 **pilot job（领航作业）**：先申请一块资源”占位”，再由自己的调度器把小任务派进去。HTCondor 的 glidein、Ray、Dask on Slurm 都是这个思路。



关键设计：

设计	做法	为什么
队列用文件夹	`pending/running/done/failed/hold` 五个目录，任务就是一个 shell 脚本	不需要数据库或消息中间件；用 `ls` 和 `mv` 就能看和改；集群上装不了 Redis/Kafka

设计	做法	为什么
取任务用 `mv`	从 pending 移到 running	同一文件系统内的 rename 是原子操作，不会被执行两次
顺序靠文件名	文件名带时间戳 `20261007-082745-xxx.sh`	先进先出；想插队就起一个更小的名字
暂停而不删除	移到 `hold/`	想恢复时移回来即可，不丢东西
空闲自动退出	60 分钟没任务就结束作业	遵守集群规定，不空占显卡
每个任务前检查服务	模型服务挂了先重启再执行	一个服务崩了不会让后面所有任务都失败

1.3 “补位”任务（backfill）：不让卡空着，又能随时让路

队列里固定放一个特殊任务 `zz-filler.sh`（名字以 zz 开头，所以总排在最后）。它按配置文件 `filler.conf` 里的顺序，**一次只跑一个实验任务**，跑完一个就检查：

- 如果 pending 里来了别的任务 → 把自己重新放回队尾，退出，让新任务先跑（**在任务边界上让路**，不打断正在跑的任务）；
- 否则继续下一个。

`filler.conf` 示例：

```
L outputs/experiment/cluster_longmem_v4_webgenlm7b_r1          config/long_memory_cluster_v4.json
L outputs/experiment/cluster_checklist_only_webgenlm7b_r1      config/long_memory_cluster_checklist_only.j
N outputs/experiment/cluster_nomem_webgenlm7b_r2
```

好处： – **改实验顺序不用停作业**：直接编辑这个文件，下一个任务就按新顺序走。 – **插队很便宜**：放一个诊断脚本到 pending，补位脚本跑完当前任务就让路。 – **可续跑**：每个任务完成后写一行到 `progress.jsonl`；补位脚本每次都从”第一个还没完成的任务”开始，作业崩了重启也不会重复做。

这在调度理论里叫 **backfill scheduling**：用低优先级的长尾工作填满空档，高优先级工作一来就让路。

1.4 失败处理

情况	处理	对应的通用概念
调用模型偶尔返回坏 JSON / 断连	同一次调用最多试 3 次，间隔递增（10s、20s）	重试 + 退避（retry with backoff）
整个任务崩了	记一笔到 `filler-failures.txt`，下次再试；同一任务失败 3 次就跳过	毒消息 / 死信队列（poison message, DLQ）
被限流（HTTP 429）	按 `Retry-After` 等待	限流处理
请求超过模型上下文	先截短代码再重试，逐级截	降级（graceful degradation）
拆清单失败两次	用代码按句子拆，作为兜底，并记录”用了兜底”	fallback + 可观测

要区分”能重试”和”不能重试”的失败：系统自己的偶发故障（网络、坏 JSON）可以重试；实验本身的结果（网站编译失败）不能重试，否则会改变实验协议、对比不公平。这一点面试很加分：重试不是越多越好，要先判断失败的性质。

1.5 资源隔离（同一台机器上跑多套服务）

- **端口偏移**：每个作业用 `WEBGEN_PORT_OFFSET`（0、20、40），所有服务端口加上偏移，两个作业可以同时在一台机器上跑。
- **进程管理目录隔离**：每个作业有自己的 `PM2_HOME`，互不干扰。
- **评测锁**：同一时间只允许一个评测占用浏览器环境（文件锁）。

1.6 踩过的坑（面试讲"我从错误中学到什么"）

1. **日志被覆盖**：补位脚本每次重启都用 `>` 写同一个日志文件，导致夜里 6 个任务崩溃的报错全丢了。改成每个任务单独一个日志文件（`tee -a runner-<id>.log`）。教训：**日志要按工作单元分开，且追加写**。
2. **监控脚本误报 / 漏报**：用空格拆分远程命令的输出，`grep -c` 在没有匹配时既打印 0 又触发 `|| echo 0`，字段错位，结果一次误报、一次漏报。改成远程直接输出 `key=value`。教训：**监控本身也要测试；结构化输出比按位置解析可靠**。
3. **改代码影响正在跑的实验**：新功能一律加开关、默认关闭，只在新配置里打开，正在跑的实验不受影响（**feature flag**）。
4. **上线前用假对象做冒烟测试**：用假的模型客户端跑通"存储→去重→检索→改写→输出"整条流程，提前发现了一个数据库约束错误（新功能一写入就报错，会让整晚实验全部白跑）。

1.7 Java 程序员怎么对照

本项目	Java 生态里的对应
文件夹队列 + worker 循环	`BlockingQueue` + `ExecutorService`；分布式下是 RabbitMQ / Kafka 消费者
原子 `mv` 取任务	数据库里 `SELECT ... FOR UPDATE SKIP LOCKED`；Redis `RPOPLPUSH` / `LMOVE`
补位任务在任务边界让路	优先级队列 `PriorityBlockingQueue`；协作式中断（检查 `Thread.interrupted()` 再退出）
失败 3 次跳过	Spring Kafka / RabbitMQ 的 DLQ、`maxAttempts`
重试 + 退避	Resilience4j `Retry`、Spring Retry `@Retryable(backoff=...)`
每任务前检查服务	Kubernetes readiness / liveness probe；Spring Boot Actuator health
`progress.jsonl` 断点续跑	Spring Batch 的 `JobRepository`（记录 step 执行状态，可 restart）
空闲 60 分钟退出	线程池 `keepAliveTime`；Serverless 的空闲回收
端口偏移、目录隔离	多实例部署时的配置隔离（profile、环境变量）
功能开关默认关	Feature toggle（Togglz、Unleash、LaunchDarkly）

1.8 面试时可以这样讲（STAR）

- **S**：共享 GPU 集群排队长、模型加载慢、每人有卡数上限，实验方案又经常要调整。
- **T**：让实验连续跑一个月不断，同时能随时插入诊断、调整顺序，不丢排队位置。
- **A**：申请一个长驻作业，加载一次模型；内部用文件夹实现任务队列，原子移动取任务；补位任务按配置跑实验、在任务边界让路；三层失败处理；按任务分日志；新功能加开关。

- **R**：一个作业连续跑了几百个评测任务，中间插入多次诊断、换了几次实验顺序都没停作业；夜里偶发崩溃自动重试后全部补齐。

常见追问：– *为什么不用 Kafka/Redis？* 集群上装不了，也不需要高吞吐；任务以分钟计，文件系统完全够用，而且可读、可手改。– *两个 worker 同时取同一个任务怎么办？* rename 是原子的，只有一个能成功；失败的那个拿下一个。– *worker 崩了正在跑的任务怎么办？* 它留在 `running/`；重启后按 `progress.jsonl` 判断有没有完成，没完成就重跑（**至少一次**语义，所以任务本身要能重复执行而不出错，即幂等）。

---

2. 其他可以讲的点（简要）

2.1 SSH 连接复用和隧道

- `ControlMaster`：一次输入密码 + 双因素认证，后面所有 ssh/rsync 复用同一条连接（约 12 小时有效）。
- `ssh -O forward -L 本地端口:127.0.0.1:远程端口`：在已有连接上动态加端口转发，把集群上的模型服务映射到本机来调试。
- 对照：跳板机、`kubectl port-forward`。

2.2 模型服务化

- 用 vLLM 把模型包装成 OpenAI 兼容的 HTTP 接口，业务代码只换 `base_url` 就能在本地模型和云 API 之间切换（**面向接口编程 / 适配器模式**）。
- 每个角色（生成、评测、记忆、向量）各自一套环境变量，不混用（**按角色隔离配置**）。

2.3 实验设计（做数据 / 算法相关岗位可讲）

- **对照组和消融**：v4 同时改了”prompt 改成清单”和”加了记忆”两件事，所以加一个”只有清单、没有记忆”的对照组，才能说清楚提升来自哪里。
- **防数据泄露**：记忆只能用之前任务的信息，不能用当前任务的评测答案；人工分析只能看开发集，不能看测试集。
- **指标口径**：曾经把”外观评分”那条记录也算进功能检查的分母，导致所有准确率都偏低；发现后统一更正并写明口径。教训：**指标先定义清楚再算，报告时注明样本量和口径**。

2.4 写给 AI/LLM 应用的工程经验

- 模型输出要当成不可信输入：解析失败要能重试或兜底，不能让整个任务崩掉。
- 小模型”听代码不听文字”：给可照抄的代码示例，比写一段建议有效得多。
- 注入的信息不是越多越好：条目太多，模型只做提到的部分（“喧宾夺主”），要控制长度、要有过滤。

---

3. 待补充

- （以后遇到新的工程问题，在这里追加。）